Honours Project Logbook

Verified bit-level parsing

Stan Kestens

Last updated: October 24, 2026

Contents

1 Rocq basics **1**

2 A parser for RISC-V I-type **1**

3 A generic format language **2**

1 Rocq basics

8 October 2026 `Exercises.v (not in this repo)`

Exercises to get used to Rocq: simple programs on `bool`, `nat` and lists, and proofs by rewriting and induction (e.g. $n + 0 = n$, $n + S\,m = S(n + m)$, $\neg\neg b = b$, and lemmas about `++` on lists). The last part introduces `take_bits`, which is reused in everything below.

Status. Several exercises are still `Admitted`.

2 A parser for RISC-V I-type

8 October 2026 `code/rv32-decode/Parser.v`

Bits are represented as `list bool`, with bit 0 first. Write $\mathbb{B}^*$ for bit strings, $u \cdot v$ for concatenation and $|u|$ for length.

Definition 2.1 (take)**.** $\mathsf{take}_n : \mathbb{B}^* \to (\mathbb{B}^* \times \mathbb{B}^*)_\perp$ splits off the first n bits: $\mathsf{take}_n(b) = (u, r)$ if $b = u \cdot r$ with $|u| = n$, and $\perp$ if $|b| < n$.

Lemma 2.2.

(a) If $|u| = n$ *then* $\mathsf{take}_n(u \cdot r) = (u, r)$. *take_bits_app*

(b) If $|u| = n$ *then* $\mathsf{take}_n(u) = (u, \varepsilon)$. *take_bits_all*

(c) If $\mathsf{take}_n(b) = (u, r)$ *then* $b = u \cdot r$. *take_bits_split*

An I-type instruction has five fields, read from bit 0 upwards:

$$\text{opcode (7)} \mid \text{rd (5)} \mid \text{funct3 (3)} \mid \text{rs1 (5)} \mid \text{imm (12)}.$$

An instruction is a 5-tuple of bit strings, and it is *well-formed* if each field has the right length. The serializer concatenates the fields; the parser takes them off one by one with take and fails if anything is left after the last field.

Theorem 2.3 (Roundtrip)**.** *parse_serialize*
If i *is well-formed, then* $\mathsf{par}(\mathsf{ser}(i)) = i$.

Theorem 2.4 (Exactness)**.** *serialize_parse*
If $\mathsf{par}(b) = i$*, then* $\mathsf{ser}(i) = b$*.*

Both proofs go field by field, using Lemma 2.2. There are also two examples: `addi x1, x0, 5` parses to the expected fields, and a 3-bit input is rejected.

Status. Done. The proofs are specific to this one format.

3 A generic format language

24 October 2026 `code/format-lib/Format1_Seq.v`

Instead of one parser per format, formats are described by a datatype, and the parser and serializer are defined by recursion on the description. The correctness proofs are then done once, for every format.

Definition 3.1 (Formats)**.**

$$f \ ::= \ \mathsf{Bits}\ n \ \mid \ \mathsf{Seq}\ f_1\ f_2 \qquad (n \in \mathbb{N})$$

$\mathsf{Bits}\ n$ is a field of n bits; $\mathsf{Seq}\ f_1\ f_2$ is f_1 followed by f_2.

Definition 3.2 (Values and well-formedness)**.** The values of a format (`denote`) are

$$[\![\mathsf{Bits}\ n]\!] = \mathbb{B}^*, \qquad [\![\mathsf{Seq}\ f_1\ f_2]\!] = [\![f_1]\!] \times [\![f_2]\!],$$

and $\mathsf{wf}_f \subseteq [\![f]\!]$ requires every field to have the right length:

$$\mathsf{wf}_{\mathsf{Bits}\ n}(v) \iff |v| = n, \qquad \mathsf{wf}_{\mathsf{Seq}\ f_1\ f_2}(v_1, v_2) \iff \mathsf{wf}_{f_1}(v_1) \wedge \mathsf{wf}_{f_2}(v_2).$$

`denote` maps a value of `fmt` to a type, so the parser and serializer have dependent types.

Definition 3.3 (Serializer and parser)**.**

$$\begin{aligned}
\mathsf{ser}_{\mathsf{Bits}\ n}(v) &= v,\\
\mathsf{ser}_{\mathsf{Seq}\ f_1\ f_2}(v_1, v_2) &= \mathsf{ser}_{f_1}(v_1) \cdot \mathsf{ser}_{f_2}(v_2),\\
\mathsf{par}_{\mathsf{Bits}\ n}(b) &= \mathsf{take}_n(b),\\
\mathsf{par}_{\mathsf{Seq}\ f_1\ f_2}(b) &= \begin{cases} ((v_1, v_2), b_2) & \text{if } \mathsf{par}_{f_1}(b) = (v_1, b_1) \text{ and } \mathsf{par}_{f_2}(b_1) = (v_2, b_2),\\ \bot & \text{otherwise.} \end{cases}
\end{aligned}$$

The theorems are stated with an arbitrary rest r after the parsed part; without it the induction step for Seq does not go through.

Lemma 3.4. *parsing_keeps_length*
If $\mathsf{take}_n(b) = (u, r)$ *then* $|u| = n$*.*

Theorem 3.5 (Roundtrip)**.** *parse_serialize*
For all f*,* v*,* r*: if* $\mathsf{wf}_f(v)$ *then* $\mathsf{par}_f(\mathsf{ser}_f(v) \cdot r) = (v, r)$*.*

Proof. Induction on f. Bits: Lemma 2.2(a). Seq: reassociate, then the induction hypothesis for f_1 with rest $\mathsf{ser}_{f_2}(v_2) \cdot r$ and for f_2 with rest r. □

Theorem 3.6 (Exactness)**.** *serialize_parse*
For all f*,* b*,* v*,* r*: if* $\mathsf{par}_f(b) = (v, r)$ *then* $b = \mathsf{ser}_f(v) \cdot r$*.*

Proof. Induction on f. Bits: Lemma 2.2(c). Seq: if both parses succeed, $b = \mathsf{ser}_{f_1}(v_1) \cdot b_1$ and $b_1 = \mathsf{ser}_{f_2}(v_2) \cdot r$ by the induction hypotheses. □

Theorem 3.7 (Closure)**.** *parse_wf*
For all f, b, v, r: if $\mathsf{par}_f(b) = (v, r)$ then $\mathsf{wf}_f(v)$.

Proof. Induction on f, using the previous lemma for Bits. □

Corollary 3.8 (I-type)**.** *itype_roundtrip*
With f_I = Seq (Bits 7) (Seq (Bits 5) (Seq (Bits 3) (Seq (Bits 5) (Bits 12)))): if $\mathsf{wf}_{f_I}(v)$ then $\mathsf{par}_{f_I}(\mathsf{ser}_{f_I}(v)) = (v, \varepsilon)$.

Proof. Theorem 3.5 with $r = \varepsilon$. □

This recovers the roundtrip theorem of `Parser.v` without any proof steps specific to I-type.

Status. Done.